2025 International Conference on Visual Analytics and Data Visualization (ICVADV)

REAL-TIME DROWSINESS DETECTION SYSTEM USING EYE ASPECT RATIO AND LIGHTWEIGHT MACHINE LEARNING

Mrs. Jinu Sophia
Department of CSE
Rajalakshmi Engineering College
Chennai,India
jinusophia.j@rajalakshmi.edu.in

Kaveeshaa S
Department of CSE
Rajalakshmi Engineering College
Chennai,India
230701146@rajalakshmi.edu.in

Krithika PV
Department of CSE
Rajalakshmi Engineering College
Chennai, India
230701157@rajalakshmi.edu.in

Abstract– Driver and operator drowsiness continues to be a major cause of accidents in the transportation and industrial sectors worldwide. This paper discusses the design and implementation of a real-time drowsiness detection system. It uses computer vision, facial landmark analysis, and machine learning to track alertness levels through a standard webcam. The system employs MediaPipe Face Mesh to extract 468 facial landmarks. From these, six key eye coordinates per eye are isolated to calculate the Eye Aspect Ratio (EAR). A Logistic Regression classifier, trained on synthetically generated EAR distributions, classifies the subject's state as either Alert or Drowsy in real time. The backend uses a Python Flask REST API that accepts base64-encoded webcam frames, performs MediaPipe-based preprocessing, computes EAR, and returns prediction results along with confidence scores. A JavaScript-based frontend dashboard shows a live webcam feed, real-time EAR values, confidence percentages, and detection logs. It also triggers audio alerts through the Web Audio API when consecutive drowsy detections occur. The system is designed to be lightweight, GPU-free, and deployable on standard hardware. Experimental results show strong responsiveness and classification accuracy, making the system suitable for practical, low-latency drowsiness monitoring applications. Future improvements may involve integrating deep learning models like CNNs and LSTMs to increase accuracy and support multi-state classification.

Keywords
Drowsiness Detection, Eye Aspect Ratio (EAR), MediaPipe Face Mesh, Logistic Regression, Flask REST API, OpenCV, Real-Time Classification, Facial Landmark Detection, Web Audio API, Driver Monitoring System, Computer Vision, Machine Learning.

I. INTRODUCTION

Drowsiness among drivers and operators poses a serious safety issue that often doesn't receive enough attention across transportation, aviation, and industrial sectors. The National Highway Traffic Safety Administration (NHTSA) reports that drowsy driving is responsible for thousands of deadly road accidents each year. Many of these incidents are not documented because it’s tough to attribute them to sleepiness after they occur. Fatigue affects cognitive abilities, slows down reaction times, and lowers awareness in ways similar to alcohol intoxication. This makes early and accurate detection crucial for public safety. Developing technology that can monitor alertness in real-time without being intrusive is an important research goal that combines computer vision, machine learning, and human-computer interaction expertise. Let's talk about how drowsiness detection works. Traditional methods focus on physiological signals like EEG, EOG, and HRV. These techniques are accurate but not very practical for everyday use because they need special equipment and controlled environments. On the other hand, vehicle-based methods look at things like steering patterns and lane deviations, offering a contact-free solution. However, these are reactive and depend on specific car systems.

Now, vision-based systems come into play by observing behaviors such as eye closure or head nodding through standard cameras. This approach allows for non-intrusive, real-time monitoring without needing any specialized gear. The Eye Aspect Ratio (EAR), introduced by Soukupová and Čech in 2016, is a metric that's both robust and efficient. It comes from the geometric relationship between vertical and horizontal distances of eye landmarks. This scalar value effectively indicates how open the eyes are.
In this paper, a comprehensive real-time drowsiness detection system is suggested. It uses Google's MediaPipe Face Mesh for extracting 468 facial landmarks efficiently. A Logistic Regression classifier runs via a Python Flask REST API, paired with a modern JavaScript dashboard on the front end.
Here's how the system works: It starts with your webcam capturing frames. These frames are then encoded in Base64 format before being sent to the Flask API. MediaPipe processes them next, followed by calculating the EAR. Finally, the machine learning model analyzes this data to detect drowsiness.

In essence, each part of this pipeline—from frame capture to ML model analysis—plays a crucial role in creating an effective detection system. Here's how it works: First, a prediction gets made. Then, JSON sends back a response. After that, the UI updates itself. Lastly, an alert goes off if needed. All this happens on regular CPU hardware without relying on deep learning accelerators. The frontend allows for live webcam feeds and shows real-time EAR and confidence levels. It also logs detection history and uses the Web Audio API for sound alerts.

979-8-3315-2139-4/25/$31.00 ©2025 IEEE

The system's design is modular to make it flexible. So, if someone wants to replace the Logistic Regression model with advanced deep learning models like CNNs or LSTMs in the future, they can do so without altering the overall system infrastructure.

II.LITERATURE REVIEW

Over the last twenty years, researchers have delved deep into understanding how to detect drowsiness and fatigue. They've come up with three main approaches: examining vehicle-related data, analyzing physiological signals, and observing behavior through vision-based techniques. Back in 2008, Picot and his team showed that EEG could accurately detect drowsiness by looking at how alpha waves change as someone gets tired. Three years later, Dong and colleagues explored using EOG systems to monitor slow eye movements linked to microsleeps. These physiological methods are spot-on clinically; however, they rely on wearable sensors, which aren't always comfortable or practical for everyday use.

Switching gears to vehicle-based methods, Liu's team in 2009 came up with a clever idea. They suggested checking for tiny adjustments in steering or noticing patterns when drivers drift out of their lanes as a way to spot fatigue. These methods act as indicators for driver fatigue. They're not intrusive, but they react to situations instead of predicting them. For effective use, they must connect with vehicle control systems, which makes it hard to use them alone. On another note, Soukupová and Čech made waves in 2016 with their study on Appearance-Based Vision Systems. They came up with the Eye Aspect Ratio (EAR) using dlib's 68-point facial landmark detector. By calculating a simple geometric ratio from eye landmarks, their work could reliably tell if eyes were open or shut. This method has since become a standard in many systems aimed at detecting drowsiness. Also worth mentioning is Redmon et al.'s YOLO framework. While its main purpose was object detection, later studies adapted it for pinpointing face regions before extracting landmarks. Enhancing the ability to handle changes in lighting is crucial. In 2017, Weng and his team introduced a Convolutional Neural Network (CNN) model that analyzes the NTHU Drowsy Driver Detection dataset, focusing on identifying the states of eyes and mouths. Then, in 2021, Pan and colleagues took it further by using Long Short-Term Memory (LSTM) networks. These networks help in understanding how facial features change over time, which makes it easier to spot when someone is slowly getting drowsy. However, these methods aren't without drawbacks; they need lots of training data, powerful GPUs, and take longer to process information. This limits their practicality for use in web browsers or devices with limited processing power.

On another note, Guo and his team came up with a different solution in 2020. They suggested a mix of techniques involving EAR-based eye analysis paired with mouth opening measurements (known as Mouth Aspect Ratio or MAR) along with estimating head position. The Kalman filter is useful for smoothing data over time. Combining multiple features has proven to be more reliable than using just one. Lately, systems based on Google's MediaPipe have become popular due to their effectiveness. In 2019, Lugaresi and colleagues unveiled MediaPipe, a framework that works across platforms and supports real-time perceptual pipelines. It can detect face landmarks on mobile devices in less than a millisecond, making it perfect for environments where resources are limited.

Browser-based and edge systems have seen growth thanks to WebRTC and JavaScript ML frameworks like TensorFlow.js. This trend has sparked interest in researching drowsiness detection that runs natively in browsers. In 2022, Raza and team showcased a client-side drowsiness monitoring tool using face-api.js. However, the smaller model for landmark detection hindered its accuracy. A better approach involves hybrid architectures: the frontend manages capturing and displaying while Python on the backend handles inference tasks. This setup strikes a good balance.

III. PROPOSED SYSTEM

A. Dataset

Creating a real-time labeled drowsiness dataset is often complicated and requires a lot of resources. Instead, this system opts for a synthetic dataset that relies on Eye Aspect Ratio (EAR) values. The system uses EAR as the primary feature. There are two class labels: "Alert" with EAR values ranging from 0.25 to 0.38, and "Drowsy" with values between 0.10 and 0.22. By generating data programmatically, it mimics real-world eye behavior efficiently. This approach results in a lightweight dataset that enables quick training and eliminates the need for large image datasets.

B. Dataset Preprocessing

Start by generating EAR values that fall within specific ranges. Next, use thresholds to assign labels. Afterward, combine the dataset and shuffle it thoroughly. Break it down into two parts: 80% goes for training, and 20% for testing. Normalization is optional and usually not necessary for Logistic Regression.

This approach brings several advantages. First, it keeps computational demands low. Plus, model training becomes faster as a result. Additionally, this method suits real-time applications quite well.

C. Model Architecture

The system being proposed uses a simple and effective model that separates alertness from drowsiness. It relies on Logistic Regression, a type of binary classification model. This setup works well in real-time situations without needing a powerful GPU, making it easy to use on basic systems. Unlike other methods that depend heavily on deep learning with images, this one focuses on features. The main feature it looks at is the Eye Aspect Ratio (EAR), which comes from measuring the geometry around the eyes. EAR is a reliable way to tell if eyes

are open or closed because its value drops when eyes shut. This makes it great for spotting when someone is getting sleepy. The model processes just one number for its analysis. You start with an EAR value and get a probabilistic output showing how likely it is that the user feels drowsy. Logistic Regression fits this task well because the data can be easily separated using specific EAR thresholds. The model doesn't require much computational power, it's easy to understand, and simple to train, which makes it perfect for real-time applications. When training, a labeled dataset gets created using synthetically generated EAR values. If the EAR value falls between 0.25 and 0.38, it's marked as Alert (0). On the other hand, values from 0.10 to 0.22 indicate Drowsiness (1). The model identifies the decision boundary between these two categories by adjusting a logistic (sigmoid) function accordingly. The function takes input EAR values and assigns them a probability score ranging from 0 to 1. The Logistic Regression model is mathematically represented as:

$$P(y=1|x) = \frac{1}{1 + e^{-(wx + b)}}$$

Here, x stands for the EAR value, w is the learned weight, and b represents the bias term. To classify whether an individual is alert or drowsy, the output probability is compared to a threshold, often set at 0.5. After training, the model gets saved in a file using the pickle (.pkl) format and linked with a Flask backend.

In real-time use, EAR values obtained from live webcam frames feed into the model for immediate predictions. The results provided by the model are... The system predicts a class label, assigns a confidence score, and calculates the EAR value. To make it more reliable, there's a feature that keeps track of drowsiness over time using a counter. Instead of just one prediction, the system observes several predictions in succession. If many frames in a row show signs of drowsiness, the system becomes more confident in its detection and sends out an alert. This method helps cut down on false alarms from brief eye movements like blinks. The model is designed to balance accuracy, clarity, and speed, making it perfect for detecting drowsiness in real-time applications. Plus, its modular setup means you can easily add advanced models later on, like CNNs or LSTMs, for better analysis of both time-based and spatial features.

D. Libraries and Framework

- Python: The core language for backend tasks and machine learning applications.

- Flask: A simple framework to create REST APIs and facilitate communication between client and server.
- OpenCV: Handles image processing and processes input frames from the frontend.

- MediaPipe Face Mesh: Detects 468 facial landmarks, specifically eye features, to calculate EAR.

- HTML: Constructs the web page layout.
- CSS: Provides styling for a modern look with responsive design.
- JavaScript: Manages webcam interactions, API requests, and updates content dynamically.

- getUserMedia: Grants access to the webcam for capturing live video.

- Canvas API: Extracts frames from video streams.

- Web Audio API: Plays alert sounds when drowsiness is detected.

E. Algorithm Explaination

The system kicks off by grabbing live video through the browser's getUserMedia API, which lets you tap into your webcam. Every so often, frames are snatched using the Canvas API and turned into Base64 images for sending over. These images head to the Flask backend via a POST request aimed at the /predict endpoint. Once they land there, the backend gets busy decoding and running them through MediaPipe Face Mesh to spot 468 facial landmarks. Out of these, six crucial points around each eye are picked to figure out the Eye Aspect Ratio (EAR). This ratio tells us how open or closed your eyes are—lower values suggest that your eyes might be shut and hint at drowsiness. This EAR value goes on to... A trained Logistic Regression model processes the data, determining whether the state is alert or drowsy. It outputs status, a confidence score, and the calculated EAR value. These prediction results arrive at the frontend in JSON format. The user interface displays them dynamically, showing visual indicators for status, EAR value, and confidence percentage.

Meanwhile, the system logs activities and updates basic statistics like total scans and drowsy events. For better reliability, it uses a mechanism to track consecutive drowsy predictions. If this streak surpasses a set limit (say two or more), an alert goes off. To notify users audibly, it employs the Web Audio API to produce sound alerts. Options include an instant mute feature and a cooldown period to avoid repetitive notifications.
This complete process supports efficient real-time monitoring of user alertness with minimal computational load.

F. System and Implementation

This setup works as a comprehensive, real-time application combining a web-based frontend with a Python Flask backend. On the frontend, it uses the browser's webcam API to capture live video. Frames get extracted periodically with the Canvas API. These frames are then encoded into Base64 and sent to the backend through REST API calls. Once the Flask server receives each frame, it decodes the image and applies MediaPipe Face Mesh for facial landmark detection. Eye coordinates are derived from these landmarks, which then help calculate the Eye Aspect Ratio (EAR). This ratio is fed into a pre-trained Logistic Regression model. Finally, based on this input, the model predicts whether the user is alert or drowsy. The system provides results and a confidence score in JSON format. On the frontend, an interactive dashboard showcases

the predictions dynamically. It features status indicators, EAR values, and confidence levels. Additionally, a log of recent detections is maintained, with basic statistics updated, like total scans and drowsy events. A streak-based mechanism watches for consecutive drowsy predictions. An alert triggers when it reaches a certain threshold. The audio alert system uses the Web Audio API to notify users, offering options such as mute control and cooldown to prevent constant alerts. The design focuses on low latency and efficient processing, making it ideal for real-time use without needing high computational power.

IV. RESULTS AND DISCUSSION

The team created a drowsiness detection system that works well in real-time using a webcam interface. The setup involved integrating both frontend and backend parts effectively, as the video frames from the browser were processed smoothly by the Flask server. With MediaPipe Face Mesh, facial landmarks were detected accurately under normal conditions, offering 468 landmark points. Among these, eye regions were consistently extracted. Using these landmarks, the Eye Aspect Ratio (EAR) was calculated, which turned out to be a dependable way to identify when eyes are closing. A Logistic Regression model was used with EAR values that were generated synthetically. This model delivered consistent performance in classifying data without requiring much computing power. The system delivers smooth predictions in real time without needing any special hardware. In tests, it showed excellent ability to spot drowsiness when the EAR values fell below a set threshold. Classification results appeared almost instantly, providing users with quick feedback.

A feature for detecting ongoing drowsy periods greatly improved the system's accuracy by cutting down on false alarms from regular blinking. Rather than responding to single frames, it examined consecutive predictions, leading to more stable decisions. The frontend dashboard presented the results effectively with status indicators like alert or drowsy, EAR values, confidence scores, and detection logs. This instant feedback made it easy for users to grasp their level of alertness clearly. The audio alert system worked smoothly, sounding alarms only when it noticed someone was really drowsy.

This clever approach stopped unnecessary interruptions by using features that pause and silence the alerts. After digging deeper, it turned out the system performed well with various users, even if their facial features were a bit different. It showed it could handle diversity quite well. Its simple design helped keep delays low and responsiveness high, which made it great for keeping an eye on things non-stop. Yet, during tests, some issues popped up. The system struggled to accurately detect facial landmarks in dim lighting or when faces were partially hidden by glasses, hands, or masks. It also had trouble when a person tilted their head too much. Quick head movements and bad camera quality didn't help either. The consistency of EAR computation took a hit. The model, which uses synthetically generated EAR values, struggles to fully reflect the variability seen in real-world datasets. This can affect classification accuracy, especially in edge cases. But despite these hurdles, the system showed overall robustness and stability. This validates that a simple, feature-based machine learning approach works effectively for real-time drowsiness detection. MediaPipe's landmark detection combined with Logistic Regression classification turned out to be efficient and easy to roll out on a large scale. The results suggest that even with minimal computing power, reliable performance can be achieved for practical uses like monitoring drivers. These insights underscore the strengths of the proposed method while pointing out areas needing improvement, such as enhancing performance under different environmental conditions and including more diverse training data for better generalization..

V. CONCLUSION AND FUTURE SCOPE

In this study, the drowsiness detection system offers an effective way to monitor alertness by leveraging the Eye Aspect Ratio (EAR) alongside a Logistic Regression model. It cleverly merges computer vision techniques with a straightforward machine learning method to identify patterns of eye closure, determining if someone is alert or sleepy. MediaPipe Face Mesh plays a crucial role here, precisely identifying facial landmarks to accurately extract eye features needed for EAR calculation. Meanwhile, using a Flask-based backend facilitates seamless communication between the frontend interface and other components. The prediction model processes data in real-time. Its standout features include quick response time, low computational demands, and easy setup. This makes it ideal for practical use without needing special equipment or high-end systems. Moreover, it has additional benefits like detecting when a person is drowsy and providing an audio alert, which boosts its reliability and ease of use. Instead of looking at predictions separately, the system uses a streak-based approach to reduce false alarms by considering patterns over time. This method results in more consistent and accurate outcomes. The frontend dashboard is crafted to be both interactive and simple to navigate, offering live updates on alertness levels, EAR values, confidence scores, and detection logs. To make the user experience better, alerts should arrive on time and be easy to understand and relevant. The system strikes a fine balance between being simple and performing well, which makes it ideal for ongoing monitoring tasks like tracking driver safety or workplace alertness. Future improvements could involve adding advanced deep learning techniques like CNNs or LSTMs to boost accuracy and capture time-based patterns more effectively. It might also benefit from new features such as multi-state classification (alert, sleepy, distracted), estimating head position, and detecting yawning to make it stronger. Moreover, using this system on mobile devices or embedded platforms like Raspberry Pi could broaden its real-world uses, especially in driver monitoring systems. Tackling issues

related to varying lighting conditions and training with real-world data would enhance its reliability and adaptability. All these upgrades could turn the system into a more comprehensive solution for real-time drowsiness detection that scales well.

REFERENCES

[1] T. Soukupová and J. Čech, "Real-Time Eye Blink Detection Using Facial Landmarks," 21st Computer Vision Winter Workshop, 2016.

[2] S. Abtahi, B. Hariri, and S. Shirmohammadi, "Driver Drowsiness Monitoring Based on Yawning Detection," IEEE International Instrumentation and Measurement Technology Conference, 2011, pp. 1–4.

[3] A. Dasgupta, S. George, and A. L. Routray, "A Vision-Based System for Monitoring the Loss of Attention in Automotive Drivers," IEEE Transactions on Intelligent Transportation Systems, vol. 14, no. 4, pp. 1825–1838, Dec. 2013.

[4] C. Dewi, R.-C. Chen, and X.-H. Liu, "Eye Aspect Ratio for Real-Time Drowsiness Detection to Improve Driver Safety," Electronics, vol. 11, no. 19, 2022.

[5] R. Ghoddoosian, M. Galib, and V. Athitsos, "A Realistic Dataset and Baseline Temporal Model for Early Drowsiness Detection," Proc. IEEE Conf. Computer Vision and Pattern Recognition Workshops (CVPRW), 2019.

[6] H. Deng and H. L. Liu, "Driver Drowsiness Detection via Deep Learning Using Convolutional Neural Networks," IEEE Access, vol. 8, pp. 129828–129837, 2020.

[7] M. S. Hossain and G. Muhammad, "Cloud-Assisted Driver Drowsiness Detection System Using Deep Learning," IEEE Cloud Computing, vol. 3, no. 4, pp. 63–71, 2016.

[8] Y. Zhang, G. Pan, J. Wu, and Z. Wang, "Driver Drowsiness Detection Based on Eye State Recognition," IEEE Intelligent Vehicles Symposium, 2017.

[9] A. Singh and P. Kumar, "Real-Time Driver Drowsiness Detection System Using Eye Aspect Ratio and Machine Learning," International Journal of Engineering Research & Technology (IJERT), vol. 9, no. 6, 2020.

[10] S. Park, F. Pan, S. Kang, and C. D. Yoo, "Driver Drowsiness Detection System Based on Feature Representation Learning Using Various Deep Networks," Asian Conference on Computer Vision (ACCV), 2016.

[

.

.